Progetto di Web Programming for Sound and Music

Relazione tecnica breve

Studente: Riccardo Cambianica
Matricola: 53147A
E-mail: riccardo.cambianica@studenti.unimi.it
A.A.: 2025/2026

Sintesi. Il progetto realizza la Track #2 come applicazione Web Audio interamente client-side. L'utente può caricare un file o registrare dal microfono, analizzare il segnale, configurare una catena di elaborazione e mastering, produrre il risultato mediante rendering offline, confrontarlo con l'originale attraverso due player e due waveform indipendenti ed esportarlo come file WAV PCM a 16 bit. L'obiettivo non è riprodurre una digital audio workstation professionale, ma integrare in un unico flusso coerente i principali concetti studiati nel corso: decodifica, nodi audio, grafi di elaborazione, registrazione, rendering offline, analisi dei campioni e generazione di file.

1. Obiettivi e scelte progettuali

L'applicazione è stata progettata attorno a tre requisiti principali. Il primo è la possibilità di usare due sorgenti equivalenti dal punto di vista della pipeline: un file WAV, MP3 o OGG selezionato dall'utente oppure una registrazione acquisita dal microfono. Il secondo è la separazione tra il segnale originale e quello processato, che devono poter essere osservati e riprodotti indipendentemente. Il terzo è l'esecuzione della catena DSP tramite `OfflineAudioContext`, in modo da produrre un nuovo `AudioBuffer` completo e successivamente esportabile.

La rappresentazione interna comune è `AudioBuffer`. Il file viene letto come `ArrayBuffer` e decodificato con `decodeAudioData()`; la registrazione produce invece chunk tramite `MediaRecorder`, uniti in un `Blob` e poi decodificati con lo stesso percorso. Questa convergenza evita di duplicare analisi, playback e processing in funzione della provenienza del segnale.

Il progetto distingue inoltre chiaramente visualizzazione e riproduzione. La libreria esterna `WaveSurfer.js` è impiegata per disegnare le waveform, mostrare il cursore temporale e ricavare il punto selezionato dall'utente. Il suono non viene però riprodotto dal player interno della libreria: il controllo resta affidato alla Web Audio API. Questa scelta consente di mantenere un'unica sorgente di verità per posizione, pausa e seek e impedisce che `WaveSurfer.js` e `AudioBufferSourceNode` avanzino in modo indipendente.

2. Architettura software

Il codice è suddiviso in moduli con responsabilità definite. `app.js` svolge il solo bootstrap: inizializza l'`AudioContext`, crea controller e viste e collega i callback principali. Lo stato condiviso conserva i buffer originale e processato, le informazioni di analisi e le impostazioni dei moduli.

Area	Responsabilità
`controllers/`	Coordinamento dei casi d'uso: ingresso audio, playback, processing ed export.
`audio/`	Gestione dell'`AudioContext`, caricamento, registrazione, player, rendering offline e codifica WAV.
`processing/`	Implementazione dei singoli stadi DSP e sanitizzazione dei relativi parametri.
`analysis/`	Calcolo di durata, sample rate, numero di canali, picco lineare e picco in dBFS.
`ui/`	Aggiornamento dell'interfaccia, manopole, controlli dei moduli e integrazione con WaveSurfer.

La separazione è importante soprattutto nel processing: il controller non contiene le formule degli effetti e il renderer non conosce i dettagli dell'interfaccia. Ogni processore espone funzioni per creare i nodi e, quando necessario, per stimare la coda temporale dell'effetto. Questo rende più semplice testare singolarmente gain, equalizzatore, compressore, distorsione, chorus, delay, riverbero, normalizzatore e limiter.

3. Flusso operativo e scelte di interfaccia

Il flusso d'uso è diviso in quattro passaggi visibili: ingresso audio, confronto delle waveform, configurazione del processing ed esportazione. Dopo il caricamento o la registrazione, i controlli del player diventano disponibili e le metriche descrivono immediatamente il segnale. Le card degli effetti possono essere abilitate o disabilitate senza perdere i valori impostati; in questo modo è possibile confrontare configurazioni diverse senza dover reinserire ogni parametro. Il pulsante di processing rappresenta esplicitamente il passaggio dal modello di impostazioni al buffer renderizzato.

Una prima versione della parte d'interfaccia dedicata all'audio processing prevedeva l'utilizzo di uno slider orizzontale per dividere i tre gruppi di card utilizzabili. Ho ritenuto più user-friendly la versione finale presente, anche grazie al feedback fornito da famigliari/amici a cui ho chiesto di utilizzare il progetto.

Originale e processato non vengono riprodotti contemporaneamente in modo automatico: i due player restano indipendenti per consentire un confronto intenzionale. La posizione viene mostrata accanto ai pulsanti Play/Pause e Stop, mentre le quattro metriche tecniche restano separate sopra il player. Il messaggio di stato comunica se il risultato è valido, se il rendering è in corso oppure se le impostazioni sono cambiate dopo l'ultima elaborazione. L'export viene abilitato soltanto nello stato coerente.

Tutte le operazioni vengono eseguite nel browser. I file scelti, la registrazione e i buffer intermedi non vengono caricati su un server. Questo riduce i requisiti infrastrutturali e mantiene il contenuto audio sul dispositivo dell'utente. L'uso di moduli locali per WaveSurfer evita inoltre dipendenze di rete durante l'esecuzione normale e facilità l'utilizzo e il test del progetto partendo dal source code.

4. Acquisizione, analisi e riproduzione

Dopo la decodifica, il buffer originale viene analizzato scorrendo tutti i campioni di tutti i canali. L'interfaccia mostra sample rate, numero di canali, picco assoluto e corrispondente valore in dBFS. Posizione corrente e durata totale sono invece aggiornate nella riga del player. L'ambito verificato è audio mono e stereo: il numero di canali viene preservato nel rendering e nell'export, ma non sono previsti routing surround, mappature 5.1/7.1 o parametri differenti per singolo canale.

La classe `AudioPlayer` gestisce un player indipendente per originale e processato.
Un `AudioBufferSourceNode` è monouso; per questo viene ricreato a ogni avvio e a ogni seek. La pausa viene simulata memorizzando l'offset già riprodotto, mentre la ripresa crea una nuova sorgente a partire da tale offset. `Stop` interrompe la sorgente e riporta la posizione a zero. Un ciclo basato su `requestAnimationFrame` legge periodicamente la posizione calcolata dal player e aggiorna sia il tempo testuale sia il playhead di WaveSurfer.

Il click sulla waveform viene convertito in un tempo espresso in millisecondi e inoltrato al player Web Audio. Durante la riproduzione il seek sostituisce la sorgente corrente senza perdere lo stato di avanzamento. Sono inoltre ignorati gli eventi `ended` provenienti da sorgenti non più attive, eventualità che può verificarsi dopo uno stop o un seek rapido.

La gestione delle interazioni utente con le due waveform tramite hover e bottoni è stata implementata seguendo la documentazione della libreria WaveSurfer.js.

5. Catena di elaborazione offline

Premendo *Apply Processing*, il sistema crea un nuovo grafo all'interno di un `OfflineAudioContext` con sample rate e numero di canali uguali a quelli della sorgente. I nodi vengono collegati nell'ordine mostrato nell'interfaccia:

`Gain` → `EQ` → `Compressor` → `Distortion` → `Chorus` → `Delay` → `Reverb` → `Normalization` → `Limiter`

5.1. Livello, equalizzazione e dinamica

L'input gain usa un `GainNode` e permette di attenuare o amplificare il segnale prima degli altri stadi. L'equalizzatore è formato da tre `BiquadFilterNode` in serie: `lowshelf` per le basse frequenze, `peaking` per la banda centrale e `highshelf` per le alte. Il compressore usa `DynamicsCompressorNode`

e rende configurabili threshold, ratio, attack e release; il knee resta quello predefinito del nodo. Tutti i valori provenienti dalla UI vengono convertiti e limitati ai range validi prima di essere assegnati agli `AudioParam`.

5.2. Effetti creativi

La distorsione utilizza un `WaveShaperNode` con una curva di soft clipping simmetrica. Il parametro drive controlla la non linearità, il mix realizza il percorso dry/wet e l'oversampling può essere disattivato oppure impostato a 2x o 4x per ridurre l'aliasing prodotto dalla trasformazione non lineare.

Il chorus riprende il modello mostrato negli esempi del corso: un breve `DelayNode` viene modulato da un `OscillatorNode` sinusoidale collegato al parametro `delayTime` attraverso un `GainNode`. Rate, depth e mix sono configurabili. Il delay/echo contiene un percorso diretto, un percorso ritardato e un anello di feedback. Il feedback è limitato sotto l'unità per evitare instabilità. Il riverbero impiega un `ConvolverNode` e una risposta impulsiva sintetica generata localmente: il rumore pseudo-casuale viene moltiplicato per un inviluppo di decadimento e la generazione deterministica rende ripetibile il risultato a parità di parametri.

5.3. Durata e code degli effetti

Poiché il rendering offline produce un buffer di lunghezza prefissata, è necessario stimare la parte di segnale che continua dopo la fine della sorgente. Il chorus aggiunge il massimo ritardo modulato; il delay calcola quante ripetizioni sono necessarie affinché l'ampiezza scenda sotto circa 0,001, cioè approssimativamente -60 dB, con un limite di sicurezza di 15 secondi; il riverbero aggiunge la durata della risposta impulsiva. Le tre code sono sommate perché gli effetti sono collegati in serie e la coda di uno stadio può attraversare quelli successivi. Quando il wet mix di un effetto è pari a zero non viene aggiunta durata, evitando silenzio inutile nel file processato.

6. Normalizzazione, limiter ed export

La normalizzazione viene applicata in un secondo passaggio offline, dopo il rendering della catena creativa. Il sistema misura il massimo valore assoluto del buffer e calcola il guadagno necessario per portarlo al target selezionato. Si tratta di *peak normalization*: non vengono calcolati RMS, LUFS o loudness percepita e il rapporto dinamico interno del segnale resta invariato.

Il limiter è l'ultimo stadio, per evitare che elaborazioni successive rialzino i picchi. Il primo passaggio usa un `DynamicsCompressorNode` con ratio 20:1, knee nullo, attack di 1 ms e release di 50 ms. Dopo il rendering, `enforcePeakCeiling()` misura il picco effettivo e, solo se necessario, applica un guadagno uniforme con un piccolo margine di sicurezza. Il metodo garantisce quindi un ceiling sample-peak deterministico, ma non rappresenta un true-peak limiter professionale capace di rilevare picchi inter-sample.

Il buffer finale viene codificato senza librerie esterne nel formato RIFF/WAVE PCM little-endian a 16 bit. L'encoder scrive l'header, conserva il sample rate, interlaccia i canali e converte i campioni floating-point nel range rappresentabile. Se il limiter è disattivato e il buffer supera il full scale, la conversione deve necessariamente limitare i sample all'intervallo $[-1, 1]$, con possibile hard clipping.

7. Coerenza dello stato e robustezza asincrona

Il rendering è asincrono e può durare sensibilmente più della modifica di un controllo. Per evitare risultati incoerenti, `ProcessingController` crea prima del job una copia profonda e congelata delle impostazioni. Tutti i passaggi del renderer usano tale snapshot e non leggono direttamente lo stato mutabile dell'interfaccia. Il flag `isProcessing` impedisce render concorrenti e disabilita temporaneamente il pulsante di applicazione.

Una revisione numerica delle impostazioni permette inoltre di rilevare modifiche effettuate mentre il job è in corso. Il risultato prodotto con lo snapshot iniziale può essere mantenuto per l'ascolto, ma viene marcato come non aggiornato e `ExportController` disabilita nome file e download finché l'utente non applica nuovamente il processing. Un identificatore del job impedisce invece che il risultato di una sorgente precedente sostituisca lo stato corrente quando viene caricato un nuovo file durante il

rendering.

8. Collegamento agli esempi e verifica

Il progetto integra e amplia gli esempi forniti dal docente per i seguenti moduli:

- `06_01_load_file.html` per caricamento e decodifica contenuto
- `07_01_gain.html` e `07_04_compressor.html` per livello e dinamica
- `09_01_reverb.html`, `09_02_echo.html` e `09_03_chorus.html` per gli effetti
- `14_03_download.html` per la generazione del file
- `15_01_audio_capture.html`, `15_02_save.html` e `15_04_offline_recorder.html` per acquisizione, registrazione e rendering offline

La differenza principale è l'integrazione di dimostrazioni isolate in un'applicazione modulare, configurabile e dotata di stato coerente. Ho cercato di rimanere più fedele possibile nell'implementazione e nella scelta dei parametri per sopperire alla mia mancanza di conoscenze pregresse nell'ambito dell'audio processing.

9. Testing

Oltre al testing "hands-on" durante lo sviluppo e le prove di utilizzo effettuate da utenti non sviluppatori, ho cercato di creare una piccola suite di test tecnici. Ho scelto il framework `Playwright` dietro consiglio di un amico sviluppatore front-end. La suite automatica contiene 43 test Playwright distribuiti su funzioni pure, nodi audio reali e flussi end-to-end. Sono verificati sanitizzazione dei parametri, neutralità e limiti degli effetti, risposta dell'EQ, code temporali, normalizzazione, ceiling del limiter, interleaving e header WAV, playback e interazione dell'interfaccia. La configurazione prevede Chromium, Firefox. In realtà sono disponibili anche i test per WebKit, non eseguiti sul mio pc in quanto il sistema operativo (Arch Linux) non supportava alcune librerie necessarie al funzionamento. Ho comunque testato in pratica il funzionamento generale del progetto tramite l'utilizzo del browser safari su un dispositivo Apple. Le pagine HTML e i fogli CSS sono stati verificati con i validator W3C con esito positivo. Restano necessariamente manuali la qualità sonora percepita, l'accesso al microfono e il comportamento delle periferiche su hardware reale.

La strategia di test è organizzata su più livelli. I test più semplici controllano funzioni deterministiche come conversione dB/lineare, calcolo del guadagno di normalizzazione, sanitizzazione dei parametri e stima delle code. Un secondo gruppo crea veri nodi Web Audio in un contesto del browser e verifica il comportamento del segnale, ad esempio che un gain unitario sia neutro, che il mix dry conservi l'ingresso e che la curva di distorsione rimanga limitata e simmetrica. I test sul renderer costruiscono buffer sintetici noti e controllano durata, picco e risposta della catena completa. Infine, gli scenari end-to-end interagiscono con l'interfaccia per verificare gli stati dei pulsanti, l'invalidazione dell'export e il flusso caricamento–processing–download.

10. Limiti e conclusioni

Il principale limite deriva da una scelta implementativa precisa: il processing non è in tempo reale. Ogni modifica richiede un nuovo rendering e non è possibile monitorare immediatamente gli effetti mentre si spostano i controlli. Questa scelta semplifica però il confronto, permette di conservare le code e produce direttamente un buffer esportabile. Un'eventuale evoluzione verso il monitoraggio real-time richiederebbe un secondo grafo connesso alla destinazione, la gestione dinamica degli `AudioParam` e una sincronizzazione diversa con l'export offline.

Il supporto dichiarato è mono/stereo e l'unico formato di esportazione è WAV PCM a 16 bit. In pratica è possibile caricare anche audio multicanale 5.1/7.1, ma per la natura del processo è impossibile effettuare le operazioni di audio processing su tutti i canali. La waveform processata viene generata a partire da una codifica WAV e rappresenta quindi la versione quantizzata destinata all'export, mentre il player riproduce direttamente il buffer floating-point. Il sistema non implementa inoltre preset, salvataggio delle sessioni o annullamento delle modifiche.